

BULKREACH

Bulk SMS & Email Platform

Comprehensive System Documentation — v2.0

Architecture · API · Database · Deployment · Data Archive · Legal & Compliance
v2.0 — Includes complete Data Archive subsystem (Sections 18–23)

Platform	BulkReach
Version	1.0
Market	Uganda / East Africa
Stack	FastAPI · Next.js · PostgreSQL · Redis · ClickHouse
New in v2.0	Data Archive subsystem — Sections 18–23
Document type	Full system spec + developer guide

1. Executive Summary

BulkReach is a multi-tenant SaaS platform that lets businesses send bulk SMS and email campaigns to up to 20,000 recipients per batch, with full per-recipient personalisation derived from uploaded contact files. Two customer types are served:

- Self-service customers — log in, upload contacts, compose, and send campaigns independently via the web dashboard.
- Managed-service clients — brief the BulkReach team, who handle everything from contact import to delivery, then receive a branded client success report as proof.

Every campaign generates two reports: an analytics PDF (delivery rates, open rates, failures, charts) and a branded client success report auto-emailed to the client.

The platform is built for Uganda / East Africa — Africa's Talking for SMS (direct MTN and Airtel Uganda connections) and Mailgun SMTP for email. Flutterwave handles payments: MTN Mobile Money, Airtel Money, Visa, and Mastercard.

Version 2.0 adds the Data Archive subsystem — a compliance-grade, superadmin-only data vault. It permanently stores all campaign history, contact records, delivery event logs, client reports, billing records, and uploaded source files across the entire platform lifetime. It includes a configurable retention policy engine, legal holds, anonymisation-based erasure, and a full access audit trail.

This document is the single source of truth for the entire platform. A developer or AI agent reading it with no other context should be able to build, deploy, and operate the full system including the archive subsystem.

2. System Architecture

2.1 Layer Overview

BulkReach is a seven-layer system. The Data Archive is an eighth independent subsystem fed by a nightly ingestion pipeline from the live platform.

Layer	Description
Client layer	Web dashboard (Next.js), Mobile app, REST API consumers, Webhook receivers
API gateway	Rate limiting, JWT auth, TLS termination, API key validation
Core services	Campaign manager, Contact importer, Template engine, Scheduler
Dispatch layer	SMS dispatcher (Africa's Talking), Email dispatcher (Mailgun SMTP), Retry engine
Analytics layer	Delivery tracker, Analytics aggregator, PDF report generator, Client report emailer
Data layer	PostgreSQL (relational), Redis (queue + cache), ClickHouse (events), S3/MinIO (files)
Data Archive	Separate PostgreSQL + ClickHouse + S3 Glacier vault. Superadmin-only. Nightly ingestion.

2.2 Full Technology Stack

Component	Technology	Version	Rationale
API framework	FastAPI	0.111+	Async-native, auto OpenAPI docs, Pydantic validation
Relational DB	PostgreSQL	16	ACID compliance for accounts, campaigns, billing
Job queue	Redis + ARQ	7 / 0.26	Fully async Redis queue — lighter than Celery
Analytics DB	ClickHouse	24.3	Columnar — handles millions of delivery events fast
SMS gateway	Africa's Talking	Latest	Direct MTN/Airtel UG connections, best East Africa delivery
Email gateway	Mailgun SMTP	Latest	Pre-warmed IPs, ~\$0.80/1k, 1k/month free tier

Component	Technology	Version	Rationale
File storage	S3 / MinIO	Latest	Object storage for uploaded contact files
Cold storage	S3 Glacier	Latest	Long-term archive for source files and Parquet exports
PDF generation	WeasyPrint	62+	HTML → PDF, no headless Chrome needed
Frontend	Next.js + shadcn/ui	14+	React, TypeScript, production-grade components
Payments	Flutterwave	Latest	MTN MoMo, Airtel Money, Visa, Mastercard — Uganda
Auth	JWT (python-jose)	3.3	Stateless, horizontally scalable
Migrations	Alembic	1.13	PostgreSQL schema versioning

2.3 Email: Mailgun SMTP vs Self-Hosted

Self-hosted Postfix is free but a fresh IP sending 20,000 emails lands in spam because Gmail, Yahoo, and MTN/Airtel mail servers rate-limit unknown IPs. Mailgun provides pre-warmed, trusted SMTP endpoints. Implementation is standard aiosmtplib pointing at smtp.mailgun.org:587 with STARTTLS — no vendor lock-in.

Option	Cost at 20k / Deliverability
Self-hosted Postfix	Free / Poor until IP warmed (2–3 months)
Gmail SMTP	Free but 500/day limit — unusable at 20k
Mailgun SMTP (chosen)	~UGX 56,000 (\$15) / Excellent — pre-warmed IPs
Amazon SES	~UGX 53,000 / Excellent — SDK setup more complex
SendGrid	~\$20/month minimum / Excellent

Note: After 2–3 months of consistent low-bounce sending your own IP builds reputation and you can switch to self-hosted Postfix at zero cost. Mailgun is the reputation shortcut while your domain is new.

3. Product Features

3.1 Contact Import

The parser extracts all columns, detects phone and email columns, and makes remaining columns available as personalisation merge tags.

Format	Parser & Behaviour
CSV (.csv)	pandas / csv module. Encoding auto-detected (chardet). Delimiter sniffed (comma, semicolon, tab, pipe).
Excel (.xlsx/.xls)	openpyxl via pandas. First sheet used by default.
Word (.docx)	python-docx. Extracts first table. Falls back to body text as CSV if no table.
PDF	pdfplumber (primary), camelot (fallback). Tables extracted from all pages, concatenated.
Comma/newline pasted	Custom split logic. No column headers — personalisation unavailable.

3.1.1 Column Detection

1. Name matching — header checked against known hints (phone, mobile, tel, email, e-mail, etc.).
2. Value sampling — if name matching fails, first 20 non-empty values tested. >50% pass = selected column.
3. All remaining columns after phone and email become available merge tags.

3.2 Personalisation

Jinja2 SandboxedEnvironment with double-brace syntax. Tags are case-insensitive. Works identically for SMS (plain text) and email (HTML + plain text, with HTML auto-escaping to prevent XSS).

Template	Rendered output for Grace Nakato
Dear {{first_name}}, your balance is UGX {{amount}}.	Dear Grace, your balance is UGX 450,000.
{{first_name}} {{last_name}} — ref {{reference}}	Grace Nakato — ref REF-00124
Invoice {{invoice_no}} due {{due_date}}	Invoice INV-2025-001 due Friday

Note: SMS has a 160-character limit (GSM-7) or 70 characters (Unicode/emoji). The composer warns users if personalisation expands a message beyond the limit. Longer messages split into multiple SMS parts and are charged separately.

3.3 Campaign Types & Dispatch

- SMS only, Email only, or Both (dispatched concurrently with `asyncio.gather`).
- Send immediately or schedule for future. Scheduler checks for due campaigns every minute.
- Up to 20,000 recipients per campaign. Sub-batches of 500 (email) or 300 (SMS) with 1-second inter-batch delay.
- Retry engine: exponential backoff, up to 3 retries per message. Failure reason recorded in delivery log.

3.4 Reports

- Analytics PDF — auto-generated after every campaign. Contains delivery stats, open/click/bounce rates, time-series chart, provider breakdown, and error report with exact failure reasons.
- Client success report — branded PDF with client logo, custom header, delivery confirmation statement, key metrics, and signed confirmation block. Auto-emailed after campaign completion.

4. Database Schema

4.1 PostgreSQL — Live Platform

Table	Purpose & Key Columns
accounts	id (UUID PK), name, email, plan, status, created_at, logo_url, report_header, accepted_terms_at, accepted_privacy_at, accepted_data_retention_at, terms_acceptance_ip, terms_acceptance_user_agent.
users	id, account_id (FK), email, hashed_password, role (owner/admin/member/superadmin), is_active, last_login_at.
campaigns	id, account_id, name, type (sms/email/both), status (draft/queued/scheduled/sending/completed/failed), sms_body, email_subject, email_html_body, email_plain_body, from_email, from_name, sms_sender_id, scheduled_at, completed_at, messages_sent, sms_sent, sms_failed, email_sent, email_failed, elapsed_seconds, error_message.
contact_lists	id, account_id, name, source_filename, source_format, total_contacts, valid_contacts, duplicate_count, columns (JSONB), phone_column, email_column, merge_columns (JSONB), checksum_sha256, created_at.
campaign_contacts	campaign_id (FK), contact_id (FK), phone (E.164), email, merge_data (JSONB).
contacts	id, list_id (FK), phone, email, raw_data (JSONB), is_valid, validation_error, created_at.
plans	id, name, price_ugx, messages_per_month, batch_size, features (JSONB).
subscriptions	id, account_id, plan_id, status, current_period_start, current_period_end, flutterwave_subscription_id.
payments	id, account_id, amount_ugx, method (mtn_momo/airtel_money/card), status (pending/successful/failed), flutterwave_tx_id, created_at.
reports	id, campaign_id, type (analytics/client_success), file_url, emailed_to, emailed_at, created_at.
managed_campaigns	id, campaign_id, brief_text, status (briefed/copy_approved/scheduled/sending/complete/report_issued), account_manager_id, approved_at.
api_keys	id, account_id, name, key_hash, last_used_at, is_active.
audit_logs	id, actor_id, actor_email, action, resource_type, resource_id, details (JSONB), ip_address, created_at. Append-only — no UPDATE or DELETE ever permitted.

4.2 ClickHouse — Live Analytics Events

```
CREATE TABLE delivery_events (  
    event_id      String, campaign_id String, account_id String,  
    channel       LowCardinality(String), -- 'sms' | 'email'  
    recipient     String,  
    status        LowCardinality(String), --  
'delivered' | 'failed' | 'opened' | 'clicked' | 'bounced'  
    provider      LowCardinality(String), -- 'africastalking' | 'mailgun'  
    message_id    String, cost String, error String,  
    created_at    DateTime64(3, 'UTC')  
) ENGINE = MergeTree()  
    PARTITION BY toYYYYMM(created_at)  
    ORDER BY (campaign_id, channel, created_at)  
    TTL created_at + INTERVAL 2 YEAR;
```

Materialised views: `campaign_stats_mv` (totals per campaign/channel/status/day) and `account_daily_stats_mv` (totals per account/channel/day for billing and analytics).

5. API Reference

5.1 Base URL & Auth

Base URL: <https://api.bulkreach.ug/api/v1> | All endpoints require Bearer JWT or X-API-Key header.

5.2–5.7 Core Endpoints Summary

Endpoint group	Key endpoints
Auth (/auth/)	POST register · POST login · POST refresh · POST logout · POST forgot-password · POST reset-password
Campaigns (/campaigns/)	GET list · POST create · GET {id} · PUT {id} · DELETE {id} · POST {id}/send · POST {id}/cancel · GET {id}/preview · GET {id}/progress (SSE)
Contacts (/contacts/)	POST upload · POST paste · POST map-columns · GET lists · GET lists/{id} · GET lists/{id}/rows · DELETE lists/{id}
Reports (/reports/)	GET list · GET {id}/download · POST {id}/send · POST campaigns/{id}/generate-report · POST campaigns/{id}/client-report
Payments (/payments/)	POST initiate · POST webhook (Flutterwave) · GET history · GET /subscriptions/current
Admin (/admin/)	GET accounts · GET accounts/{id} · PUT accounts/{id}/plan · PUT accounts/{id}/status · GET managed-queue · POST managed/{id}/approve · GET revenue · GET health · GET queue-health · GET audit-log

5.8 Archive API Endpoints (/admin/archive/) — Superadmin only

Endpoint	Description
GET /dashboard	KPI summary: total campaigns, contacts, delivery events, storage GB.
GET /campaigns	Search archived campaigns. Query: q, account_id, type, date_from, date_to, page.
GET /campaigns/{id}	Full archived campaign: message body snapshot, counters, contact list summary.
GET /campaigns/{id}/export	Download campaign as CSV/Excel: all recipients, delivery status, timestamps.

Endpoint	Description
POST /campaigns/{id}/restore	Re-queue archived campaign as new live campaign for re-send.
GET /contacts	Search contact master registry. Query: q (phone or email), account_id, campaign_id.
GET /contacts/{id}	Full contact record with all campaigns they appeared in.
POST /contacts/{id}/anonymise	Replace phone/email with SHA-256 hash, null name fields. Logs to audit trail.
GET /delivery-logs	Query archive ClickHouse. Query: campaign_id, channel, status, date_from, date_to.
GET /reports	All archived PDFs with metadata.
GET /reports/{id}/download	Download PDF from S3 or initiate Glacier retrieval (12hr delay).
GET /billing	Archived payment and billing records. Query: account_id, date_from, date_to.
GET /files	Source file manifest. Query: account_id, format, storage_class (standard/glacier).
GET/PUT /retention-rules	List or update platform-wide retention rules.
PUT /retention-rules/account/{id}	Set per-account retention override.
GET /legal-holds	All active and historical legal holds.
POST /legal-holds	Place a legal hold. Body: { resource_type, resource_id, reason }.
DELETE /legal-holds/{id}	Lift a legal hold. Reason required. Logged immutably to audit trail.
GET /erasure-requests	Pending and resolved erasure requests.
POST /erasure-requests/{id}/execute	Execute anonymisation for an erasure request.
GET /export	Generate full account data ZIP for GDPR subject access or client offboarding.
GET /access-log	Who accessed the archive, what they viewed, downloaded, or modified, and when.

6. Backend Code Structure

6.1 Directory Layout

```
backend/app/
├── core/          config.py · database.py · security.py · dependencies.py
├── api/v1/        auth · campaigns · contacts · reports · payments · admin
├── api/v1/archive/
│   ├── dashboard · campaigns · contacts · delivery_logs · reports · billing
│   ├── files · retention · legal_holds · erasure · export
├── models/        account · campaign · contact · billing · archive
├── schemas/       Pydantic request/response models
├── services/
│   ├── parsers/   contact_parser.py · template_engine.py
│   ├── dispatch/  email_dispatch.py · sms_dispatch.py
│   ├── reports/   pdf_generator.py · client_report.py
│   └── archive/
│       ├── ingestion.py · retention_engine.py · anonymiser.py
│       └── glacier.py · export_builder.py
└── workers/       campaign_worker.py · report_worker.py · archive_worker.py
```

6.2 Key Services

Service	Responsibility
contact_parser.py	Accepts CSV/Excel/Word/PDF/pasted text. Returns ParseResult: contacts (phone E.164, email normalised, raw_data dict), all column names, detected phone/email columns, merge_columns list, valid/duplicate/error counts, up to 50 row-level error messages.
template_engine.py	SandboxedEnvironment Jinja2 renderer. render_sms(), render_email_body(), render_subject(), extract_merge_tags(), validate_template_tags(), preview_render(). Case-insensitive tags. HTML auto-escaped for email.
email_dispatch.py	Async Mailgun SMTP bulk sender. One connection per campaign, reused for all recipients. Batches of 500 with 1s delay. Per-recipient rendering. Inline CSS (premailer). List-Unsubscribe header. Fires on_result callback for SSE progress streaming.
sms_dispatch.py	Africa's Talking bulk SMS. Groups contacts by rendered message (identical messages share one API call). Sub-batches of 300. Includes sms_length_info() GSM-7/Unicode counter for composer warnings.
campaign_worker.py	ARQ background job. Loads campaign + contacts from PostgreSQL. Runs SMS and email dispatch concurrently. Streams delivery events

Service	Responsibility
	to ClickHouse. Updates campaign progress in real time. Enqueues report generation job on completion.
archive/ingestion.py	Nightly ARQ job at 02:00 EAT. Queries live PostgreSQL for completed campaigns older than ARCHIVE_LIVE_RETENTION_MONTHS. Copies rows to archive schema with archived_at timestamp. Exports aged ClickHouse events to Parquet on S3.
archive/retention_engine.py	Evaluates all retention rules daily. Checks each domain (campaigns, contacts, delivery events, reports, billing, files) against its rule. Moves files to Glacier, deletes expired records, skips records under legal hold. Logs every action to audit trail.
archive/anonymiser.py	Executes right-to-erasure requests. Replaces phone with SHA-256(phone), email with SHA-256(email), nulls first_name/last_name/raw_data fields. Preserves row, campaign links, and delivery statistics. Logs action immutably.
archive/glacier.py	Manages S3 lifecycle transitions. Initiates Glacier retrieval for download requests (standard 12-hour retrieval). Checks retrieval status. Updates file manifest with storage_class and retrieval_available_at.
archive/export_builder.py	Generates a ZIP file for GDPR subject access requests or client offboarding. Contains: campaigns CSV, contacts CSV, delivery logs CSV, all PDFs, billing CSV, and a manifest JSON. Streams to S3 for download.

7. Frontend Structure

7.1 Technology

- Next.js 14+ (App Router), TypeScript, shadcn/ui (Radix UI + Tailwind CSS)
- Zustand (global state), React Query (server state), React Hook Form + Zod (forms)
- Recharts (analytics), react-dropzone (file upload), EventSource SSE (live progress)

7.2 Route Structure

Route	Layout / Purpose
/ (marketing)	Public landing page. No auth required.
/(auth)/login and /signup	Auth pages. Signup records accepted_terms_at on account creation.
/dashboard	ClientLayout. Campaign list, stats, compose, contacts, reports, settings.
/admin	AdminLayout. Superadmin operational dashboard. Sidebar includes 'Data archive' link under a 'Data' section label at the bottom.
/admin/archive	ArchiveLayout (completely separate layout). Replaces AdminLayout sidebar with archive sidebar. Header shows '← Back to admin' link.
/admin/archive/campaigns	Campaign search and browse.
/admin/archive/campaigns/[id]	Campaign detail with restore button.
/admin/archive/contacts	Contact master registry search and anonymise.
/admin/archive/delivery-logs	Delivery event explorer.
/admin/archive/reports	PDF vault.
/admin/archive/billing	Billing records.
/admin/archive/files	Source file manifest with storage class.
/admin/archive/retention	Retention rules manager.
/admin/archive/legal-holds	Legal hold management.
/admin/archive/erasure	Erasure request queue.
/admin/archive/export	Full account data ZIP export.
/admin/archive/access-log	Archive access audit trail.

7.3 Navigation: Admin to Archive

AdminLayout and ArchiveLayout are completely separate Next.js layout files. Clicking 'Data archive' in the main admin sidebar navigates to /admin/archive and replaces the entire layout. The archive header shows '← Back to admin' which navigates back to /admin. This is pure Next.js nested routing — no state management or toggles needed.

The 'Data archive' item sits under a 'Data' section label at the bottom of the main admin sidebar — visually separated from operational sections. Implementation: AdminLayout = app/admin/layout.tsx. ArchiveLayout = app/admin/archive/layout.tsx. Both check role === 'superadmin' via a shared requireSuperadmin() middleware.

8. Payment Integration

8.1 Supported Methods

Method	Notes
MTN Mobile Money	Flutterwave mobile_money_uganda API. Customer receives USSD prompt to confirm.
Airtel Money	Same flow — Flutterwave handles Airtel API differences internally.
Visa / Mastercard	Flutterwave inline checkout. 3DS supported.
Recurring subscriptions	Flutterwave subscription plans for monthly auto-renewal.

8.2 Payment Flow

4. Client selects plan and payment method on pricing page.
5. Frontend calls POST /payments/initiate with method and phone (mobile money).
6. Backend creates Flutterwave transaction with UUID tx_ref.
7. Mobile money: USSD push to client. Card: inline payment link returned.
8. Flutterwave fires POST /payments/webhook with status.
9. Backend verifies HMAC-SHA256 with FLUTTERWAVE_WEBHOOK_SECRET.
10. Success: activate subscription in PostgreSQL, update account plan, send confirmation email.

8.3 Managed Service Billing

Proforma invoice workflow: admin creates quote → system emails invoice PDF → client pays → webhook confirms → admin approves in managed queue → campaign dispatched.

9. Landing Page & Admin UI

9.1 Landing Page

Section	Content
Navbar	Logo, Features · How it works · Pricing · FAQs links, Log in, Get started CTA.
Hero	'Reach 20,000 people in minutes — or let us do it for you'. Two CTAs: Start free trial / Request managed service.
Stats strip	20k+ messages per batch · 98.4% delivery rate · PDF auto report every campaign.
Features	6-card grid: Contact import (with format chips), SMS + Email, Analytics & PDF, Retry engine, Client reports, Schedule & batch send.
How it works	Steps 01–04 (self-service) + Managed-only card + Every plan card.
Pricing	Starter UGX 50k · Growth UGX 150k (featured) · Business UGX 400k · Managed custom. Payment row: MTN MoMo / Airtel Money / Visa/Mastercard.
Testimonials	Three cards with quote, name, role.
FAQ	6 items: formats, self-service vs managed, client reports, trial, payments, retry.
CTA banner	Final conversion block.
Footer	Privacy · Terms · Contact · API docs · Kampala, Uganda.

9.2 Superadmin Dashboard

Component	Content
Sidebar	220px. Logo + Superadmin badge. Sections: Overview, Clients (accounts, subscriptions, managed queue with badge), Revenue, Platform, System, Data ('Data archive' link at bottom).
Topbar	Page title, breadcrumb, period toggles, notification icon, avatar.
KPI cards	Total clients · Revenue (month) · Messages today · Managed queue (pending warning). Each has 12-bar sparkline.
Panels	Recent signups + Managed queue with approve/view/edit actions for pending campaigns.
Revenue bars	Starter · Growth · Business · Managed — UGX amounts with progress bars.

Component	Content
Provider health	SMS providers (Africa's Talking + Twilio) · Email providers · Queue health (Redis, Retry, ClickHouse).
Clients table	Client · Plan · Messages/month · Status · Balance · Reports issued · Last active · Actions.

10. Infrastructure & Deployment

10.1 Docker Compose Services

Service	Image / Port / Purpose
postgres	postgres:16-alpine. Port 5432. Live relational data.
postgres_archive	postgres:16-alpine. Port 5433. Archive relational data — separate instance.
redis	redis:7-alpine. Port 6379. Job queue + caching.
clickhouse	clickhouse/clickhouse-server:24.3. Ports 8123/9000. Live analytics, 2yr TTL.
clickhouse_archive	clickhouse/clickhouse-server:24.3. Ports 8124/9001. Archive analytics, 7yr TTL or never.
minio	minio/minio:latest. Ports 9010/9001. Dev S3 — simulates Standard + Glacier tiers.
api	Custom Dockerfile. Port 8000. FastAPI + Uvicorn. Hot-reload in dev.
worker	Same Dockerfile. ARQ worker: campaign jobs + archive ingestion + report jobs.
frontend	Custom Dockerfile. Port 3000. Next.js dev server.

10.2 Environment Variables

Variable	Description & Example
SECRET_KEY	Random 32+ byte hex. Generate: openssl rand -hex 32
DATABASE_URL	postgresql+asyncpg://bulkreach:pw@localhost/bulkreach
ARCHIVE_DATABASE_URL	postgresql+asyncpg://bulkreach:pw@localhost:5433/bulkreach_archive
REDIS_URL	redis://localhost:6379/0
CLICKHOUSE_HOST / CLICKHOUSE_ARCHIVE_HOST	localhost (live) and localhost (archive, port 9001)
MAILGUN_API_KEY / MAILGUN_DOMAIN	key-xxxx / mg.bulkreach.ug
MAILGUN_SMTP_USER / MAILGUN_SMTP_PASSWORD	postmaster@mg.bulkreach.ug / Mailgun SMTP password

Variable	Description & Example
AT_API_KEY / AT_USERNAME / AT_SENDER_ID	Africa's Talking credentials / BulkReach
FLUTTERWAVE_PUBLIC_KEY / SECRET_KEY / ENCRYPTION_KEY / WEBHOOK_SECRET	From dashboard.flutterwave.com
S3_BUCKET / S3_ARCHIVE_BUCKET / S3_GLACIER_BUCKET	bulkreach-uploads / bulkreach-archive / bulkreach-glacier
S3_ENDPOINT_URL	http://localhost:9010 for dev MinIO, blank for AWS S3
ARCHIVE_INGESTION_HOURS	2 (nightly at 02:00 EAT)
ARCHIVE_LIVE_RETENTION_MONTHS	6 (move campaigns older than 6 months to archive)

10.3 Production Deployment

- API + Worker: Two separate AWS EC2 / ECS tasks. Auto-scale workers on Redis queue depth.
- PostgreSQL (live): AWS RDS Multi-AZ. PostgreSQL (archive): Separate RDS instance, read-heavy — smaller class acceptable.
- Redis: AWS ElastiCache Redis.
- ClickHouse: Self-hosted on dedicated instance. Archive ClickHouse can share the same server as a separate database with extended TTL.
- S3 (live files): AWS S3 Standard. Archive files: S3 Standard-IA. Cold files: S3 Glacier Instant Retrieval (auto-transitioned via S3 lifecycle policy after 12 months — no code required).
- Frontend: Vercel (zero config for Next.js).
- Nginx: Reverse proxy for SSL termination and rate limiting.

11. Developer Quick Start

11.1 Prerequisites

- Docker Desktop (or Docker Engine + Docker Compose v2)
- Node.js 18+ and npm
- Python 3.11+
- Git

11.2 Setup

11. Clone: `git clone https://github.com/your-org/bulkreach.git && cd bulkreach`
12. Create backend/.env from backend/.env.example — fill in all required values.
13. Start infrastructure: `cd infra/docker && docker compose up -d`
14. Install Python deps and run migrations: `cd backend && pip install -r requirements.txt && alembic upgrade head`
15. Run archive migrations: `alembic -c alembic_archive.ini upgrade head`
16. Start API: `uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload`
17. Start worker (separate terminal): `arq app.workers.campaign_worker.WorkerSettings`
18. Start frontend: `cd frontend && npm install && npm run dev`

API docs: <http://localhost:8000/docs> | Frontend: <http://localhost:3000> | MinIO console: <http://localhost:9001> | ClickHouse live: <http://localhost:8123/play> | ClickHouse archive: <http://localhost:8124/play>

11.3 Creating the First Superadmin

```
python scripts/create_superuser.py --email admin@bulkreach.ug --password yourpassword
```

12. Managed Service Workflow

The managed service moves through six states in the managed_campaigns table:

Status	Who acts and what happens
briefed	Client submits brief (objective, audience, send date). Superadmin receives notification.
copy_approved	Superadmin reviews brief, writes/edits message copy, marks approved. Client may see preview for sign-off.
scheduled	Campaign created in system with contacts imported. Send time confirmed.
sending	ARQ worker dispatching messages. Progress visible in managed queue panel.
complete	All messages dispatched. Delivery stats finalised.
report_issued	Analytics PDF generated and client success report emailed to client. Invoice marked paid.

Note: Always verify payment is confirmed before approving a managed campaign. Check the payments table or Flutterwave dashboard before clicking Approve.

13. Security Considerations

13.1 Authentication & Authorization

- JWT tokens expire after 24 hours. Refresh tokens in httpOnly cookies.
- Roles: owner, admin, member (account level) and superadmin (platform level).
- All superadmin and archive endpoints check role === 'superadmin' via FastAPI dependency.
- API keys are bcrypt-hashed before storage. Raw key shown once on creation only.
- Archive access is additionally logged to archive_access_log — every view, download, and action.

13.2 Data Security

- Uploaded contact files stored in S3 with server-side AES-256 encryption.
- Mailgun SMTP uses STARTTLS (port 587) — all email traffic encrypted in transit.
- Flutterwave webhooks verified with HMAC-SHA256 using FLUTTERWAVE_WEBHOOK_SECRET.
- Archive database uses the same PostgreSQL encryption at rest as the live database.
- All archive S3 buckets have server-side encryption and versioning enabled.

13.3 Template Injection Prevention

- Jinja2 SandboxedEnvironment prevents code injection via message templates.
- HTML email templates use autoescape=True — XSS via merge tags is impossible.

13.4 Rate Limiting

- Per-IP and per-account rate limits via Redis sliding window counter.
- Maximum 1 active campaign per account at a time.
- File upload: max 50MB, max 10 uploads/hour per account.
- Authentication: max 5 failed logins per 15 minutes per IP.

14. Pricing Model

14.1 Subscription Plans

Plan	Price	Messages/month	Key features
Starter	UGX 50,000/month	5,000	CSV + comma-paste, basic analytics, PDF delivery report, email support.
Growth	UGX 150,000/month	50,000 (20k/batch)	All file formats, scheduling, retry, branded client reports, priority support.
Business	UGX 400,000/month	Unlimited	All Growth features, multi-user team, REST API + webhooks, white-label branding.
Managed service	Custom per project	Up to 20k/campaign	Full campaign management, dedicated account manager, branded client report, SLA.

14.2 Pay-Per-Use Overage

UGX 20 per SMS and UGX 15 per email for usage beyond monthly plan limit. Charged at start of next billing cycle.

14.3 Free Trial

500 free messages on signup. No credit card required. Trial expires after 14 days or 500 messages. Full Growth plan features during trial.

15. Third-Party Services & Setup

15.1 Africa's Talking (SMS)

19. Create account at africastalking.com. Use Sandbox for development.
20. For production, register Sender ID 'BulkReach' — approval takes 3–5 days.
21. Top up balance — Uganda SMS costs ~UGX 70–100 per message.
22. Configure delivery report callback URL to `/api/v1/webhooks/sms-dlr`.

15.2 Mailgun (Email)

23. Create account at mailgun.com. Add and verify sending domain (`mg.bulkreach.ug`) — requires MX, SPF TXT, DKIM TXT DNS records.
24. Get API key from Settings > API Keys. Get SMTP credentials from Domain Settings (different from API key).
25. Configure webhook endpoint for delivery, open, click, bounce, and spam events to `/api/v1/webhooks/email-events`.

Note: Mailgun free tier: 1,000 emails/month. Essentials plan (\$35/month): 50,000 emails. At \$0.80/1k, 20,000 emails costs ~\$16 (UGX 59,000).

15.3 Flutterwave (Payments)

26. Create business account at dashboard.flutterwave.com. Complete KYC.
27. Get API keys from Settings > API. Use test keys for development.
28. Configure webhook: Settings > Webhooks > `https://api.bulkreach.ug/api/v1/payments/webhook`.
29. Copy webhook secret hash to `FLUTTERWAVE_WEBHOOK_SECRET` in `.env`.

15.4 DNS Configuration

Record	Name	Value
A	<code>api.bulkreach.ug</code>	API server IP
A	<code>bulkreach.ug</code>	Frontend server IP or Vercel IP
CNAME	<code>www.bulkreach.ug</code>	<code>bulkreach.ug</code>
MX	<code>mg.bulkreach.ug</code>	<code>mx1.mailgun.org</code>
TXT	<code>mg.bulkreach.ug</code>	<code>v=spf1 include:mailgun.org ~all</code>
TXT	<code>krs._domainkey.mg.bulkreach.ug</code>	DKIM key from Mailgun domain settings

16. Glossary

Term	Definition
ARQ	Async Redis Queue — Python background job library used for campaign dispatch and archive ingestion workers.
Archive ingestion	Nightly ARQ job that copies completed campaign data from the live platform to the archive database.
Batch size	Number of messages per API call to the provider. Default: 500 email, 300 SMS.
Campaign	Single bulk send operation targeting a contact list with a specific message.
ClickHouse	Open-source columnar analytics database used for delivery event storage.
Contact master registry	Archive table containing every phone/email ever uploaded across all accounts — deduplicated, retained permanently.
DLR	Delivery Receipt — webhook callback from SMS provider confirming handset delivery.
E.164	International phone number format standard (e.g. +256772123456). All phones stored in this format.
Glacier	AWS S3 Glacier — low-cost cold storage for infrequently accessed files. 12-hour retrieval time.
GSM-7	7-bit SMS encoding. 160 characters per part. Messages with Unicode drop to 70 chars/part.
Legal hold	A flag placed on archive records that suspends all retention policies — nothing is deleted while a hold is active.
Mailgun	Email delivery service used via SMTP. Pre-warmed IPs for high deliverability.
Managed service	BulkReach team handles the full campaign on behalf of the client.
Merge tag	Placeholder in message template (e.g. {{first_name}}) replaced with contact's row data.
ParseResult	Structured output of contact_parser.py: contacts, column names, valid/error counts, detected columns.
Retention engine	Service that evaluates retention rules daily and executes scheduled moves, transitions to Glacier, and purges.
Right to erasure	A data subject's request to have their PII removed. Handled by anonymisation (SHA-256 hash) not deletion.
SSE	Server-Sent Events — browser API for streaming real-time campaign progress from server to client.

Term	Definition
WeasyPrint	Python HTML-to-PDF library used for analytics and client success report generation.

17. Future Roadmap

Phase 2

- WhatsApp Business API integration for bulk WhatsApp messaging.
- Contact segmentation — filter contacts by column values before sending.
- A/B testing — split contact list, send two variants, compare engagement.
- Unsubscribe management — global suppression list per account.
- Link tracking — tracked redirects in emails for per-link click measurement.

Phase 3

- Multi-region deployment — East Africa nodes for lower latency (Kenya, Tanzania).
- Dedicated IP pools for high-volume accounts.
- Custom domain sending — clients send email from their own domain with custom DKIM/SPF.
- Zapier / Make.com integration — trigger campaigns from external events.
- CRM integrations — Salesforce, HubSpot contact sync.
- Enterprise SSO — SAML / OAuth for corporate accounts.

DATA ARCHIVE SUBSYSTEM

Sections 18–23

Compliance-grade data vault · Superadmin only · Nightly ingestion pipeline

18. Data Archive — Overview

18.1 What the Archive Is

The Data Archive is not a history tab or a read-only view of the live platform. It is a completely separate database and storage system that receives data from the live platform via a nightly ingestion pipeline. It has three distinct jobs:

- Long-term storage — everything the live platform eventually moves out of hot storage is archived here, structured and queryable forever.
- Legal evidence layer — tamper-evident proof that a campaign was sent, to whom, when, and exactly what message was sent, in perpetuity.
- Data sovereignty — the platform's authoritative record of every person who ever interacted with it, regardless of whether the account that uploaded them still exists.

The archive is accessible only via `/admin/archive` routes and only to users with role `=== 'superadmin'`. No client, no regular admin, and no API key can access archive data. Every action in the archive is logged to an immutable access log.

18.2 Why a Separate Database

The live platform is optimised for write performance and low latency. Campaign queries, contact imports, and delivery event streaming need fast writes. The archive is optimised for read performance, long-term storage efficiency, and compliance. Mixing them in the same database would create index bloat, backup complexity, and GDPR complications because the retention rules for live and archive data are completely different.

The archive PostgreSQL instance can run on a smaller, cheaper machine because it receives no campaign dispatch traffic. The archive ClickHouse instance can have a 7-year or never TTL without inflating the live ClickHouse storage that only needs 2 years.

18.3 Access from the Admin Portal

In the main superadmin portal sidebar, 'Data archive' appears as the only item under a 'Data' section label at the very bottom of the navigation — visually separated from all operational sections. Clicking it navigates to `/admin/archive`, which uses `ArchiveLayout` instead of `AdminLayout`. The entire sidebar replaces — the operational nav disappears and the archive nav appears. The archive sidebar header shows a '← Back to admin' link that returns to `/admin`.

This is a full context switch, not a nested panel. The two portals feel like distinct tools even though they share the same authentication session. Implementation uses Next.js nested layouts — no JavaScript state management or toggles required.

19. Archive Data Domains

The archive stores six data domains. Each is described below with its exact contents, the storage system it uses, and its retention behaviour.

19.1 Campaign Archive

Stored in: Archive PostgreSQL

- Campaign record: name, type, status, created_at, completed_at, elapsed_seconds.
- Full SMS body text exactly as sent — immutable snapshot taken at archive time.
- Full email HTML body, plain text body, subject line, from name, from email, sender ID.
- Final delivery counters: sent, delivered, failed, sms_sent, sms_failed, email_sent, email_failed.
- account_id and account_name — retained even if the account is later deleted.
- archived_at timestamp — when the ingestion job moved this campaign to archive.

19.2 Contact Master Registry

Stored in: Archive PostgreSQL (contacts table) + Archive S3 (original uploaded files)

- Every phone number (E.164) and email address ever uploaded across all accounts — globally deduplicated.
- Full name and all merge columns from the original uploaded file.
- source_filename, source_format, upload_date, account_id, account_name.
- first_seen_at and last_seen_at timestamps across the platform.
- campaign_appearances — array of campaign IDs this contact appeared in (many-to-many).
- anonymised_at — if anonymisation has been executed, set to timestamp. phone and email replaced with SHA-256 hashes, name fields nulled.
- Retained permanently. Never hard-deleted, only anonymised on erasure request.

19.3 Delivery Event Log

Stored in: Archive ClickHouse (longer TTL than live — 7 years or never)

- One row per message attempt — every single SMS and email send ever made on the platform.
- Columns: event_id, campaign_id, account_id, channel, recipient, status, provider, message_id, cost, error, created_at.
- For campaigns older than 2 years, live ClickHouse TTL expires and records survive only in archive ClickHouse.
- For very old data, events are additionally exported to Parquet files on S3 for cost-efficient long-term storage.

19.4 Reports & PDFs

Stored in: Archive PostgreSQL (metadata) + Archive S3 (PDF files)

- Every analytics PDF ever generated — stored in S3 with the original generation timestamp.
- Every client success report ever issued — stored in S3.
- Metadata: campaign_id, report type, generated_at, emailed_to, emailed_at, file_url, file_size_bytes.
- Download audit: every download logged — who downloaded, from what IP, at what time.
- Not deletable via UI. Considered legal evidence. Only removable via direct DB operation with mandatory audit log entry.

19.5 Billing & Account Records

Stored in: Archive PostgreSQL

- Every payment ever made: amount_ugx, currency, method, status, flutterwave_tx_id, created_at.
- Every plan change: from_plan, to_plan, changed_by, changed_at, reason.
- Every invoice generated with PDF snapshot path in S3.
- Account lifecycle events: created, plan changed, suspended, reactivated, deleted — all timestamped.
- All users ever associated with an account: name, email, role, active/inactive dates.
- Minimum 7-year retention for financial compliance. Cannot be reduced below 7 years.

19.6 Uploaded Source Files

Stored in: Archive S3 (Standard for 12 months, then auto-transitioned to Glacier)

- Original CSV, Excel, Word, and PDF files exactly as uploaded — unmodified.
- Linked to the contact list and campaign that used them via file manifest in archive PostgreSQL.
- SHA-256 checksum stored at upload time for tamper detection — checksum record retained in PostgreSQL even after file deletion.
- S3 lifecycle rule automatically transitions files to Glacier Instant Retrieval after 12 months.
- Glacier retrieval: standard 12-hour window. Archive UI shows retrieval_available_at once initiated.
- Default purge schedule: never. Superadmin can set per-account schedules of 24, 36, or 60 months.

20. Retention Policy Engine

20.1 Retention Rules Table

Data domain	Default retention	Override options	Deletion behaviour
Contact master registry	Never delete	Superadmin can flag individual records for erasure	Anonymised not deleted — phone/email replaced with SHA-256(original), name fields nulled. Row, campaign links, and statistics preserved.
Campaign archive	Never delete	Superadmin can set per-account purge window (minimum 24 months)	Campaign record, message body, and counters retained. Individual recipient rows may be anonymised separately.
Delivery event logs	2yr live ClickHouse, then Parquet on S3 indefinitely	Extend to 7yr or never. Minimum 1yr. Archive ClickHouse can hold 7yr+.	Events older than live TTL expire from live ClickHouse. Archive ClickHouse + S3 Parquet copy kept indefinitely unless superadmin purges.
Reports & PDFs	Never delete	None — legal evidence	Not deletable via UI. Only via direct DB operation with mandatory audit log entry.
Billing & account records	7 years (financial compliance)	Extend only — cannot reduce below 7 years	Hard-deleted after 7 years unless legal hold is active.
Uploaded source files	12mo S3 Standard → Glacier indefinitely	Set auto-purge at 24, 36, 60 months or never	Glacier objects purged on schedule. SHA-256 checksum record kept in PostgreSQL even after file deletion.
Archive access log	Never delete	None — tamper-evident, immutable	Append-only table. No DELETE or UPDATE ever permitted.

20.2 Special Rules

Rule	Behaviour
Legal hold	Any record can be placed under legal hold by the superadmin. While active, all retention policies are suspended — nothing is auto-deleted

Rule	Behaviour
	regardless of schedule. Hold requires a reason and is itself logged immutably to the audit trail.
Unsubscribed account	When a client unsubscribes or is deleted from the live platform, their account record is soft-deleted (<code>is_active = false</code> , <code>deleted_at</code> set). All campaign data, contacts, and billing records are retained indefinitely in the archive. Foreign keys to the account remain intact.
Anonymisation vs deletion	When a contact exercises a right-to-erasure request, the system anonymises: replaces phone/email with SHA-256(original) and nulls <code>first_name</code> , <code>last_name</code> , and <code>raw_data</code> fields. The row count and campaign linkage are preserved so delivery statistics remain accurate. Hard deletion is never used for contact records.
Policy acceptance tracking	Every account signup logs: <code>accepted_terms_at</code> , <code>accepted_privacy_at</code> , <code>accepted_data_retention_at</code> , <code>terms_acceptance_ip</code> , <code>terms_acceptance_user_agent</code> . This creates the legal basis for retaining data post-unsubscription as disclosed in the platform policy.
Never-delete flag	The superadmin can set a <code>never_delete</code> flag on any individual record or account. This overrides all scheduled purges for that record permanently.

20.3 Retention Engine — Daily Job

The `retention_engine.py` service runs as a daily ARQ job at 03:00 EAT (one hour after archive ingestion). It evaluates every retention rule in sequence:

30. Load all active retention rules from the archive database.
31. For each domain, query records that are past their retention window.
32. Skip any record with an active legal hold.
33. Execute the appropriate action: move to Glacier, export to Parquet, anonymise, or hard-delete.
34. Log every action taken (including skips due to legal holds) to the immutable audit trail.
35. Update the archive dashboard storage statistics.

21. Archive Ingestion Pipeline

21.1 How Ingestion Works

The `archive_worker.py` ARQ job runs nightly at 02:00 EAT. It is the only process that writes to the archive database — all other archive operations are read-only.

36. Query live PostgreSQL for campaigns with status = 'completed' and completed_at older than ARCHIVE_LIVE_RETENTION_MONTHS (default 6 months).
37. For each campaign: copy the campaign record, all campaign_contacts rows (with merge_data), the contact_list record, and the linked reports to the archive PostgreSQL schema with an archived_at timestamp.
38. Export the campaign's delivery events from live ClickHouse to a Parquet file on archive S3, then insert them into archive ClickHouse.
39. Copy source files referenced by archived contact lists from live S3 to archive S3, then apply Glacier lifecycle rule (files older than 12 months transition automatically — no code needed).
40. Upsert contacts into the global contact master registry — phone/email as unique key. Update campaign_appearances array. Do not overwrite anonymised records.
41. After successful archive: optionally soft-delete the campaign from live PostgreSQL (sets archived = true, keeps a stub with campaign_id and archive reference). Contact lists are retained in live DB until user deletes them manually.
42. Log ingestion summary (campaigns archived, contacts upserted, events exported, files copied) to audit trail.

21.2 Ingestion Configuration

Config variable	Default / Notes
ARCHIVE_INGESTION_HOUR	2 (02:00 EAT)
ARCHIVE_LIVE_RETENTION_MONTHS	6 (archive campaigns completed > 6 months ago)
ARCHIVE_BATCH_SIZE	100 (campaigns per ingestion run — prevents timeouts)
ARCHIVE_CLICKHOUSE_TTL_YEARS	7 (or 0 for never)
ARCHIVE_GLACIER_TRANSITION_DAYS	365 (S3 lifecycle rule on archive S3 bucket — set in bucket policy, not code)

Note: The archive ingestion job is designed to be idempotent — running it twice on the same data will not create duplicates. Campaigns are checked for existing archive records before insertion. Contact master registry uses upsert with phone/email as the unique key.

22. Archive UI — All Screens

22.1 Archive Sidebar Navigation

The archive sidebar (ArchiveLayout) contains the following sections and items:

Section	Nav items
Overview	Dashboard
Data domains	Campaigns · Contact registry · Delivery logs · Reports & PDFs · Billing records · Source files
Policy	Retention rules · Legal holds · Erasure requests
Tools	Restore campaign · Export archive · Access log

22.2 Screen-by-Screen Specification

Screen	Content & Key Actions
Dashboard	KPI cards: total archived campaigns, unique contact records, total delivery events, archive storage GB. Revenue breakdown by plan (archived billing data). Recent archived campaigns list. Retention rule status summary. Active legal holds count. Pending erasure requests count.
Campaigns	Search bar (searches campaign name and message body text). Filters: account, type (SMS/email/both), date range, status. Results table: campaign name, account (shows 'deleted account' if account no longer active), type, recipients, delivered %, archived date, actions (View, Export CSV, Restore). Pagination.
Campaign detail	Full campaign record. Message body snapshot (read-only). Delivery counter breakdown. Contact list summary. Link to delivery logs for this campaign. Download analytics PDF button. Restore campaign button (creates new live campaign from this archive record).
Contact registry	Search by phone number or email address. Filter by account, campaign, date first seen, date last seen. Results table: phone, email, name, first seen, last seen, campaigns appeared in, anonymised status. Click row to see full contact detail including all campaign appearances.
Contact detail	Full contact record. All personalisation data (or [anonymised] labels if anonymised). List of all campaigns this contact appeared in with delivery status for each. Anonymise button (with confirmation dialog requiring typed confirmation). Erasure request history for this contact.

Screen	Content & Key Actions
Delivery logs	Filter: campaign, channel (SMS/email), status (delivered/failed/opened/clicked/bounced), provider, date range. Results table from archive ClickHouse: recipient, channel, status, provider, message ID, cost, error reason, timestamp. Export to CSV. Note: for files older than 2 years, data sourced from archive ClickHouse or Parquet files.
Reports & PDFs	All generated reports. Filters: type (analytics/client_success), campaign, account, date range. Table: report name, type, campaign, account, generated date, emailed to, file size. Download button — if file is in Glacier, shows 'Initiate retrieval' with 12hr wait notice. Re-email button to send report to any address.
Billing records	All payment and invoice records. Filters: account, method, status, date range. Table: account, amount UGX, method, status, Flutterwave TX ID, date. Plan change history table. Invoice download buttons.
Source files	File manifest. Filters: account, format (CSV/Excel/Word/PDF), storage class (Standard/Glacier). Table: filename, format, account, upload date, linked campaign, size, storage class, checksum, actions. Glacier files show 'Restore from Glacier' button if needed.
Retention rules	Platform-wide rules table (see Section 20.1). Per-account override section: search accounts, set override retention period or never-delete flag. 'Purge preview' button shows what would be deleted if retention is run now (dry-run mode).
Legal holds	Active holds table: resource type, resource ID, reason, placed by, placed at. Lift hold button (requires typed reason, logged immutably). Historical holds table (all holds ever placed and lifted).
Erasure requests	Pending requests queue: contact ID/phone/email, requested by, requested at, status. Execute anonymisation button. Completed requests history with executed_by and executed_at timestamps.
Restore campaign	Search archived campaigns. Select a campaign to restore. Choose: re-send to original contact list, or upload a new contact list. Set new send time. Click Restore — creates a new campaign record in live PostgreSQL with status = 'draft'. Superadmin reviews and sends from main admin portal.
Export archive	Select an account (or all accounts). Choose data domains to include. Choose format (CSV/Excel/ZIP with PDFs). Click Generate — triggers export_builder.py ARQ job. Download link sent to superadmin email when ready. Intended for GDPR subject access requests and client offboarding.
Access log	Immutable log of every action in the archive UI: who accessed what screen, what they searched, what records they viewed, what they downloaded, what actions they executed. Columns: actor email, action type, resource type, resource ID, timestamp, IP address. Export to CSV.

23. Legal, Policy & Compliance

23.1 Why Data is Retained After Unsubscription

The legal basis for retaining client and contact data after an account is closed or unsubscribed is established at the point of account creation. Every signup records a timestamped, IP-logged acceptance of three policy documents: Terms of Service, Privacy Policy, and Data Retention Policy.

The Data Retention Policy explicitly states that the platform retains operational data, campaign history, contact data uploaded by the user, and billing records for the purposes of platform operation, analytics, legal compliance, and service verification — and that this retention continues after account closure. The user agrees to this policy as a condition of using the service.

Key clause to include in the Data Retention Policy: 'We retain data relating to your account, including campaign history, contact lists you have uploaded, delivery records, and billing information, for as long as is necessary for operational, legal, and analytical purposes. This retention continues after your account is closed or your subscription is cancelled, as permitted by applicable law and as disclosed in this policy, which you accepted at the time of account creation.'

23.2 Required Policy Documents

Document	Key clauses to include
Privacy Policy	What personal data is collected (account holder data, contact data uploaded by users). How it is used (campaign delivery, analytics, billing). Who it is shared with (Africa's Talking, Mailgun, Flutterwave). How long it is retained. User rights (access, correction, erasure — with caveat that erasure is by anonymisation not deletion for delivery integrity).
Data Retention Policy	Specific retention periods for each data domain (see Section 20). Explanation of why each period applies. Statement that data is retained after account closure. How users request erasure. How legal holds work. Contact email for data requests.
Terms of Service	Acceptable use for bulk messaging (opt-in requirements for recipients, anti-spam obligations). Prohibited use: sending to purchased lists, sending without recipient consent. User responsibility for ensuring their contact lists are compliant with Uganda Communications Commission regulations and applicable law.
Cookie & Consent Notice	Cookies used (session, analytics). Acceptance of Terms of Service, Privacy Policy, and Data Retention Policy at signup — with explicit 'I agree' checkboxes (not pre-ticked). Record of acceptance timestamped and stored in accounts table.

23.3 Consent Recording at Signup

The accounts table stores four consent fields populated at the moment of account creation:

```
accepted_terms_at      TIMESTAMP WITH TIME ZONE NOT NULL
accepted_privacy_at    TIMESTAMP WITH TIME ZONE NOT NULL
accepted_data_retention_at  TIMESTAMP WITH TIME ZONE NOT NULL
terms_acceptance_ip    INET NOT NULL
terms_acceptance_user_agent TEXT NOT NULL
```

These fields are set server-side from the registration request and cannot be modified after account creation. They constitute the legal record of consent and are archived permanently — even after account deletion.

23.4 Right to Erasure Handling

When a data subject submits a right-to-erasure request (e.g. a recipient who appeared in a bulk campaign), the erasure flow is:

43. Superadmin receives the request and logs it in the erasure_requests table with the contact's phone or email.
44. Superadmin searches the contact master registry to locate the record.
45. Review: check if the contact is under a legal hold. If yes, erasure cannot be executed until hold is lifted.
46. Execute anonymisation via POST /admin/archive/erasure-requests/{id}/execute — this calls anonymiser.py.
47. anonymiser.py replaces phone with SHA-256(phone), email with SHA-256(email), nulls first_name, last_name, and raw_data fields.
48. The row, campaign_appearances array, and delivery statistics are preserved — only PII is removed.
49. The erasure action is logged immutably to the archive access log with executed_by, executed_at, and the SHA-256 hashes.
50. Erasure request marked as completed with timestamp.

Note: Hard deletion of contact records is not used because it would corrupt delivery statistics — campaign sent counts, delivered counts, and aggregate analytics would become inaccurate. Anonymisation satisfies erasure requirements while preserving data integrity.

23.5 Uganda-Specific Compliance

Bulk messaging in Uganda is regulated by the Uganda Communications Commission (UCC). Key obligations:

- Recipients must have opted in to receive bulk messages. Senders (the clients of BulkReach) are responsible for maintaining opt-in records. The Terms of Service must explicitly place this obligation on the client.

- The Sender ID must be registered with the UCC and the relevant mobile operators before use.
- Messages must include an opt-out mechanism (e.g. 'Reply STOP to unsubscribe').
- Data Protection and Privacy Act 2019 (Uganda) applies to personal data processing. The Privacy Policy must comply with its requirements including lawful basis for processing, data minimisation, and data subject rights.
- Financial records must be retained for 7 years under the Companies Act and financial regulations.

23.6 Archive Access Security

Because the archive contains the most sensitive long-term data on the platform, access is restricted and logged more strictly than the live admin portal:

- Only superadmin role can access any /admin/archive route — enforced at both API and frontend layer.
- Every page view, search query, record view, file download, and mutative action (anonymise, place hold, lift hold, execute erasure) is logged to the immutable archive_access_log table.
- The archive_access_log table is append-only — no UPDATE or DELETE is ever permitted.
- Failed access attempts (non-superadmin trying to access archive routes) are logged to the main audit_logs table with action = 'archive_access_denied'.
- Superadmin sessions accessing the archive are logged with the session_id so access can be correlated with specific login events.

End of Document — BulkReach System Documentation v2.0

BulkReach Platform | Kampala, Uganda | bulkreach.ug | 23 Sections | Sections 18-23: Data Archive Subsystem